

combining dfas, operators, and nfa

in the previous lecture we learned how to design dfas given a language. how can we design a dfa that accepts many languages?

combining dfas

to design a dfa that accept many languages, we first separate the larger language into smaller languages, and then combine them. let's see an example.

example 1

problem: design a dfa M such that its language is the set of strings that contains 11 as a substring *and* ends with 0.

solution: we can split this larger language into two smaller languages (sets). the first set is the set of strings that contain 11. we can design the first dfa A like the following:

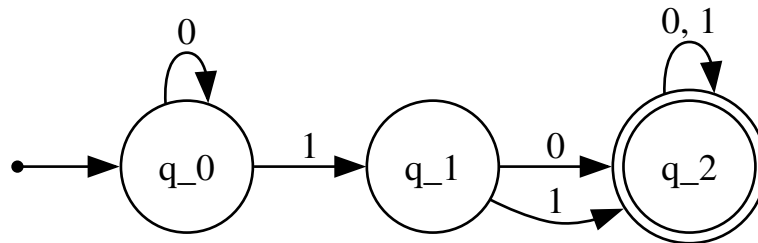


Figure 1: graphs/02_combine_ex1p1.gv

then, we design the dfa B for the second language, where it is the set of strings that end with 0.

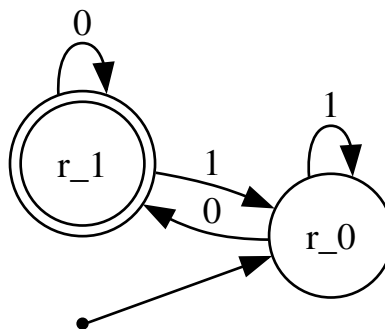


Figure 2: graphs/02_combine_ex1p2.gv

to combine these small dfas together, we first enumerate all possible states for A and B in tuples (the order does not matter):

- (q_0, r_0)
- (q_1, r_0)
- (q_2, r_0)
- (q_0, r_1)
- (q_1, r_1)
- (q_2, r_1)

this is the set of all possible states (Q) of M . you might have noticed it is a cartesian product of the states for the dfas A and B ; where $Q_M = Q_A \times Q_B$.

then we redraw the transition functions, where an

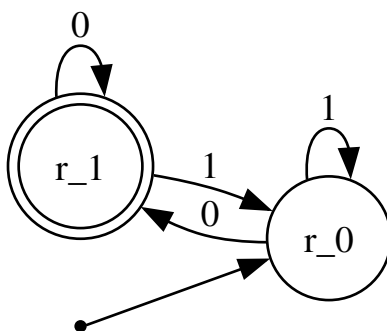


Figure 3: graphs/02_combine_ex1p3

combining dfas, formally

given a dfa $B = (Q_B, \Sigma, \delta_B, q_B, F_B)$

regular languages

a language B is **regular** if there exists a dfa A that recognizes it ($L(A) = B$).

given regular languages A and B (recognized by dfas M_A and M_B), $A \cup B$ is a **regular language**.

language operators

assume the languages A and B .

- **union:** $A \cup B = \{w | w \in A \vee w \in B\}$
- **concatenation:** $A \cdot B = AB = \{xy | x \in A \wedge y \in B\}$
- **kleene star:** $A^* = \{x_1 x_2 \dots x_k | k \geq 0 \wedge x_i \in A\}$
 - $\epsilon \cup A \cup AA \cup AAA \cup AAAA \cup \dots$ (ϵ is the empty string)

non-deterministic finite automaton (nfa)

in a dfa, we are always able to determine the current and the next state of the automaton, given an input. in nfa, however, we would not be able to know the state of the automaton until it has finished processing. for example:

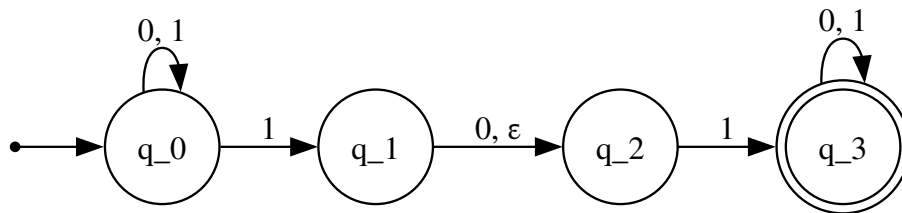


Figure 4: graphs/02_nfa.gv

todo: add abnormalities compared to dfa

in order to follow the state of a nfa, we draw a computational tree:

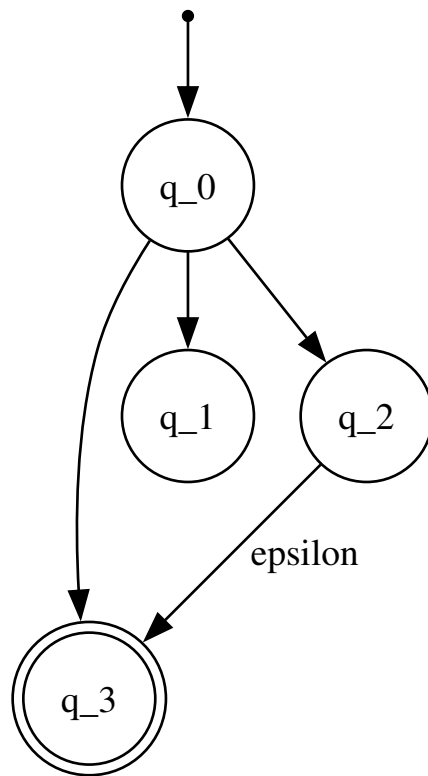


Figure 5: graphs/02_tree.gv